Sensiq: Title

Technical Reports: CL-2026-42, July 2026

Simon Völkl, Johannes Schieder, Sebastian Rosner and Andre Tien Vu
Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

***Abstract*—**
***Index Terms*—**

I. Introduction

Purpose: Establish context, define the scope of the report, and give the reader the vocabulary needed to follow the remainder of the document. This section MUST resolve every non-trivial abbreviation on its first occurrence (e.g., MCU, IoT, MQTT, TLS, AWS, REST, MVP) so that page 1 remains self-contained for a non-specialist reader.

A. Mission Statement

One concise paragraph stating WHAT the Sensiq system is, WHO it is built for (laboratory operators, facility managers, IT administrators), and the single overarching goal: reliable, low-latency environmental monitoring with scalable historical analysis on a serverless backend.

B. Motivation

Describe the practical problem: laboratories and production facilities require stable ambient conditions (temperature, humidity, gas exposure, fire risk) to guarantee experimental validity, product quality, and occupational safety. Argue why existing local-only or vendor-locked solutions are insufficient (no real-time API access, limited audit trail, per-user licensing, restricted extensibility). Conclude with the rationale for an MCU + serverless cloud approach.

C. Document Structure

Brief roadmap paragraph: cross-reference each subsequent section so the reader can navigate the report.

II. Related Work

Position Sensiq against existing academic and commercial solutions. Discuss two reference works in depth (IoT lab-safety monitoring; performance of serverless IoT pipelines) and contrast them with the design decisions of this project. Close the section with a comparison table summarising key differentiators (real-time alerts, long-term audit capability, custom API/UI, cost efficiency, laboratory focus). Template available at end of document: tab:comparison.

III. Technical Concept

High-level, technology-agnostic view of the system. This section introduces the conceptual building blocks and the end-to-end data flow. Detailed component-level descriptions follow in the Hardware, AWS Services, and User Interaction sections.

A. Technical Building Blocks

Enumerate the logical components of the system without binding them yet to concrete products: edge sensing node, secure transport channel, ingestion gateway, validation stage, hot-path store (live data), cold-path store (history), query API, notification channel, and user interface. Briefly justify the chosen programming languages (C++ firmware, Python lambdas, TypeScript infrastructure, React UI).

B. System Architecture

Present the architecture diagram (see template: fig:cloud-architecture) and walk the reader through the data flow: ESP32 -> MQTT/TLS -> AWS IoT Core -> validation Lambda -> dual persistence (DynamoDB for live, S3 + Athena for history) -> REST endpoints -> frontend / SNS.

IV. Hardware

Detailed description of the edge tier of the system.

A. ESP32 Sensor Node

Describe the ESP32 microcontroller, the selected sensors (DHT11 for temperature/humidity, KY-026 flame sensor, KY-028 thermistor, and any additional gas/light sensors), wiring, and on-device pre-processing. Include a photograph or schematic of the assembled node (see template: figure*).

B. MQTT Publication to AWS IoT Core

Explain the firmware-side MQTT client: topic structure, JSON payload schema , publish frequency, and the TLS / X.509 certificate-based authentication that secures the link between device and AWS IoT Core.

V. AWS Services

Detailed description of the cloud-side processing pipeline. Two functional routes are exposed by the backend: a low-latency live route and a long-term history route. Each is described in its own subsection together with the AWS services it relies on.

A. Live Route

Path from IoT Core rule -> validation Lambda -> DynamoDB (latest reading per device) -> API Gateway -> GET /live. Discuss latency targets, payload shape, and error handling.

B. History Route

Path from IoT Core / validation Lambda -> S3 (Parquet, partitioned by date) -> Glue catalogue -> Athena -> history Lambda -> API Gateway -> GET /history. Discuss partitioning strategy, query patterns, and expected query latency.

VI. User Interaction

Describes how end users consume the data and receive alerts.

A. Email Notifications via SNS

Explain the alert pipeline: threshold evaluation in the validation Lambda, publication to an SNS topic, and email delivery to subscribed recipients. Cover debounce behaviour and message content.

B. Web Frontend

Describe the React-based dashboard hosted on AWS Amplify: live tiles, historical charts, sensor status indicators, and authentication via AWS Cognito. Include a screenshot or mock-up (see template: figure*).

VII. Evaluation

A. Testability

Discuss per-service testability: Lambda unit tests (SAM CLI), local emulation of DynamoDB / S3 / API Gateway, limitations for IoT Core (Device Advisor only), and the role of LocalStack for higher-level integration tests.

B. Cost Estimation

Present the monthly operating-cost estimate per device under realistic assumptions (users, active hours, message volume). Compare Free Tier and Post Free Tier costs across all involved AWS services. Template available at end of document: tab:aws-costs.

VIII. Summary and Future Work

Recap the contribution of the report: a working, MCU-based IoT architecture for environmental monitoring with secure ingestion, dual-path persistence, and a user-facing dashboard. Outline next steps: extended sensor coverage, predictive analytics on historical data, multi-tenant support, and hardening for production deployment.